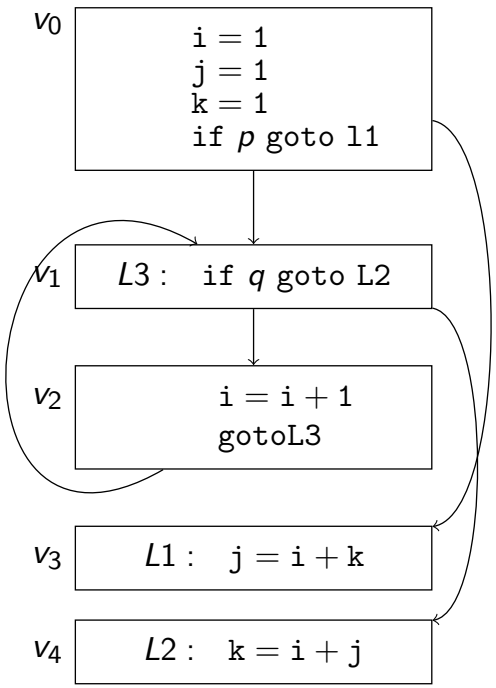
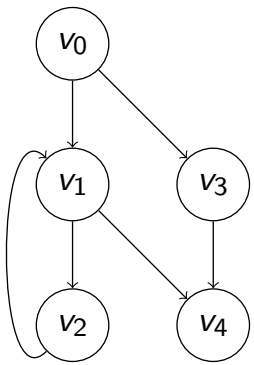
コンパイラ (12)

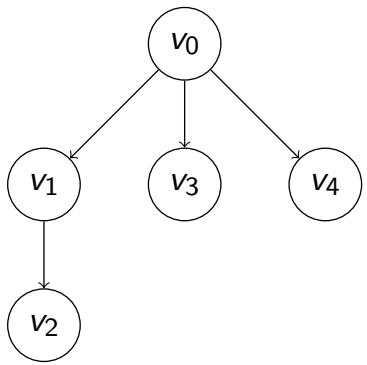
2026/11/13

フローグラフ → 支配木





フローグラフ



支配木

静的単一代入を用いた最適化

データフロー解析のもうひとつの手法:
静的単一代入 (Static Single Assignment:SSA)
各変数の定義がプログラム中で唯一
二度目の代入が発生すると変数名を変更
　値が同じの共通部分式が判別できる=式として一致
定義ごとに変数に添字をつける。合流する場合は、φ 関数で表現
いかにして、SSA 形式を得るか？

(1) φ 関数の挿入
(2) 変数の添字の決定

　　SSA 形式において最適化

(3) SSA から逆変換する。(φ 関数の消去)

近年のコンパイラでよく用いられる手法

例

```
scanf("%d",&x);
a=1;
b=2;
c=a+b;
if(x>0) a=3;
d=b;
c=a+b;
```

⇒

```
scanf("%d",&x0);
a0=1;
b0=2;
c0=a0+b0;
if(x>0) a1=3;
a2=φ(a0,a1);
d0=b0;
c1=a2+b0;
```

```
t=1
n=10
l1:  t=t*n
n=n-1
b=n>1
if b goto l1
f=t
```

⇒

```
    t1=1
    n1=10
l1: t2=φ(t1,t3)
    n2=φ(n1,n3)
    t3=t2*n2
    n3=n2-1
    b1=n3>1
    if b1 goto l1
    f1=t3
```

φ 関数の挿入

$\mathrm{x}=\varphi(\mathrm{x},\cdots,\mathrm{x})$ の挿入：

- 変数 x に対して定義 *7.1* の条件を満たす v_0，u_0，v がフローグラフに存在し，頂点 v の基本ブロックに φ 関数が含まれていないならば，v に $\mathrm{x}=\varphi(\mathrm{x},\ldots,\mathrm{x})$ を挿入する．
- 定義合流条件を満たす v_0，u_0，v がなくなるまで，上記を繰り返す．

支配辺境

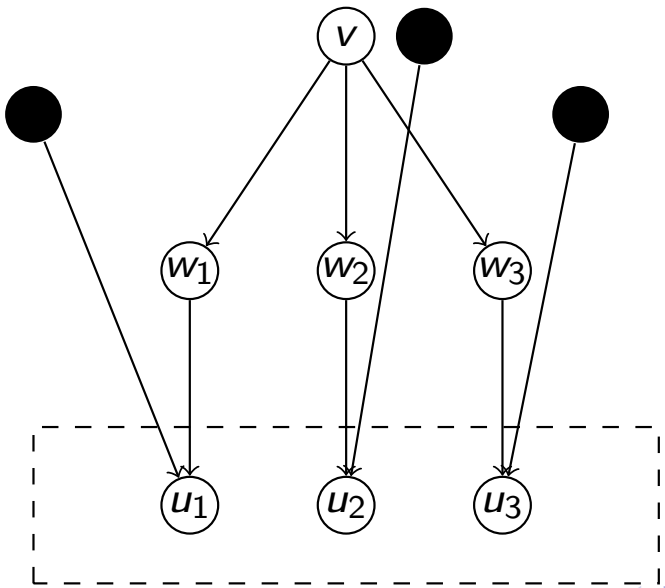
フローグラフ $G = (V, E)$

定義

v の支配辺境 *(Dominance Frontier)*

$$DF(v) = \{u \in V | \exists w.w \in pred(u), v \gtrdot w, v \not\gg u\}$$

強く支配していれば ϕ 関数を挿入する必要なし



支配辺境の計算

フローグラフ $G = (V, E, v_{init})$

補題

$$DF(v) = DF_{local}(v) \cup \bigcup_{w \in Child(v)} DF_{up}(w)$$

ここで、

$$DF_{local}(v) = \{u | v \to u, u \not\ll v\}$$

v の後続頂点で
v を経由しない根からのフローが存在する頂点

$$DF_{up}(v) = \{u \in DF(v) | u \not\ll \mathrm{idom}(v)\}$$

v の親頂点からも支配されない頂点集合

アルゴリズム＝支配木の葉から DF を計算していく

補題

1. $DF_{local}(v) = \{u | v \to u, \mathrm{idom}(u) \neq v\}$

2. v **の子頂点** w **に対して，**

   $DF_{up}(w) = \{u \in DF(w) | \mathrm{idom}(u) \neq v\}$

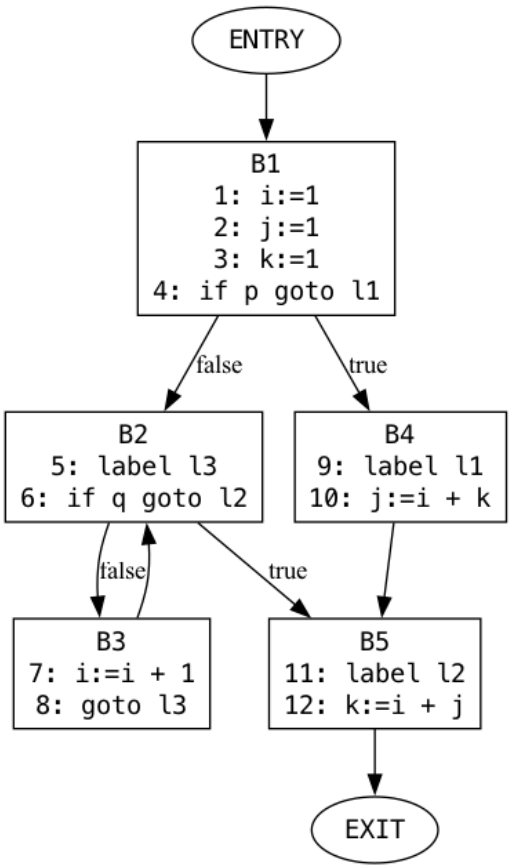
支配辺境の計算アルゴリズム

procedure COMPUTEDF(n)
  $S \leftarrow \{\}$
  for all $y \in \mathrm{succ}[n]$ **do** ▷ DF_{local} の計算
    if $\mathrm{idom}(y) \neq n$ **then**
      $S \leftarrow S \cup \{y\}$
    end if
  end for
  for all $c \in \mathrm{Child}(n)$ **do** ▷ DF_{up} の計算
    computeDF(c) ▷ 支配木の子頂点から計算する
    for all $w \in DF[c]$ **do**
      if $w \notin \mathrm{dom}(n)$ or $n = w$ **then**
        $S \leftarrow S \cup \{w\}$
      end if
    end for
  end for
  $DF[n] \leftarrow S$
end procedure

支配辺境の計算例

講義ページに python による実現を掲載

```
i:=1
j:=1
k:=1
if p goto l1
label l3
if q goto l2
i:=i + 1
goto l3
label l1
j:=i + k
label l2
k:=i + j
```



$$
\begin{aligned}
DF(B1) &= \{\} \\
DF(B2) &= \{B2, B5\} \\
DF(B3) &= \{B2\} \\
DF(B4) &= \{B5\} \\
DF(B5) &= \{\}
\end{aligned}
$$

定義合流条件の計算

定義合流条件'(S) 二つのパス $p(= v_1, \cdots, v_k)$, $q(= u_1, \cdots, u_l)$

(1) $k \geq 2$, $l \geq 2$
(2) $v_1, u_1 \in S$
(3) $v_k = u_l = v$
(4) $\{v_1, \cdots, v_{k-1}\} \cap \{u_1, \cdots, u_{l-1}\} = \emptyset$

$J(S) = \{v |$ 上の条件を満たす $v\}$

変数 x が定義される場所の集合を S とすると $J(S)$ は、x に対して最初に ϕ 関数 $x = \phi(x, x)$ が挿入される場所

ϕ 関数によって新たに定義が挿入されるため、さらに挿入が必要: 次の漸化式で不動点 $J^+(S_0)$ を計算

$$\begin{aligned} S_{i+1} &= J(S_0 \cup S_i) \\ J^{n+1}(S_0) &= J(S_0 \cup J^n(S_0)) \\ J^+(S_0) &= \lim_{n \leftarrow \infty} J^n(S_0) \end{aligned}$$

支配辺境と合流条件

定理

$DF(S) = \bigcup_{v \in S} DF(v)$ **のとき、**

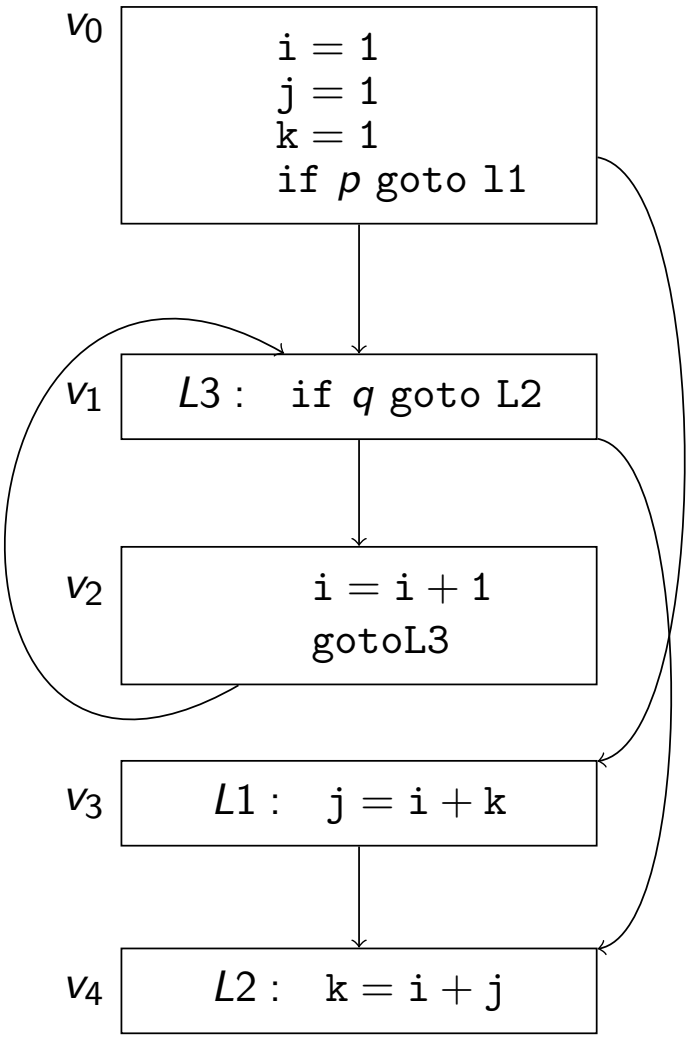
$$F_{i+1} = DF(S_0 \cup F_i) \quad (i \geq 0, F_0 = \emptyset)$$

とするとき， $DF^+(S_0) = \lim_{n \leftarrow \infty} F_n$ **とする．このとき，**

$$DF^+(S_0) = J^+(S_0)$$

合流条件は支配辺境から計算可能

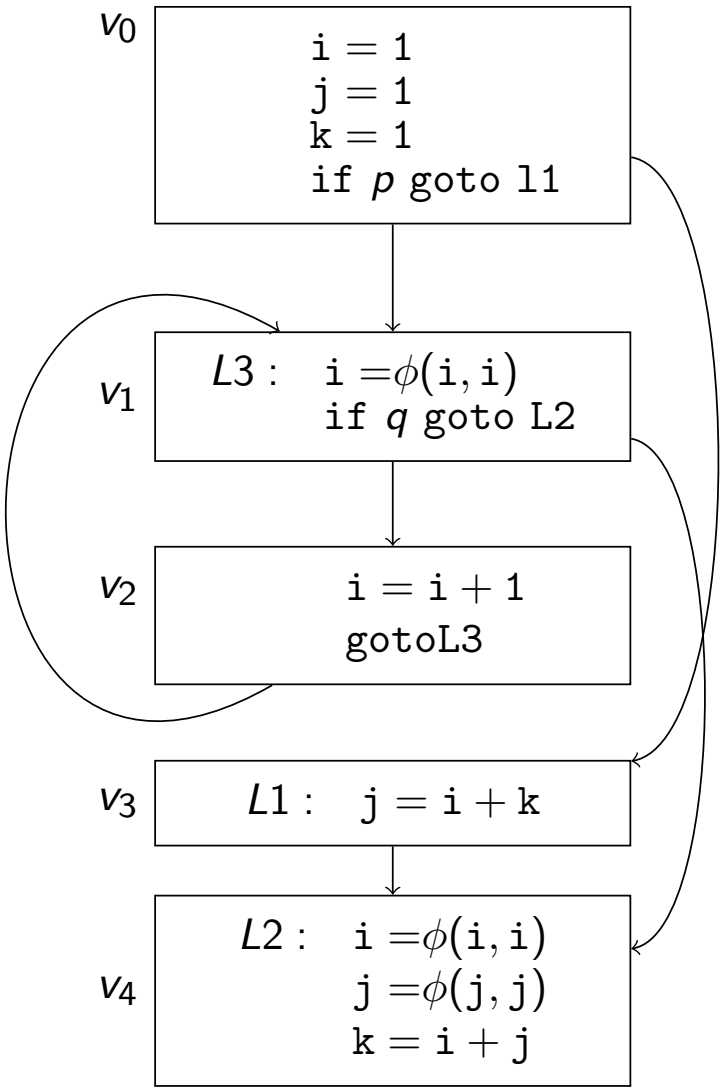
例 7.15



i の定義：$S_0 = \{v_0, v_2\}$
j の定義：$S_0' = \{v_0, v_3\}$
k の定義：$S_0'' = \{v_0, v_4\}$

$$DF^+(S_0) = \{v_1, v_4\}$$
$$DF^+(S_0') = \{v_4\}$$
$$DF^+(S_0'') = \emptyset$$

例 7.15



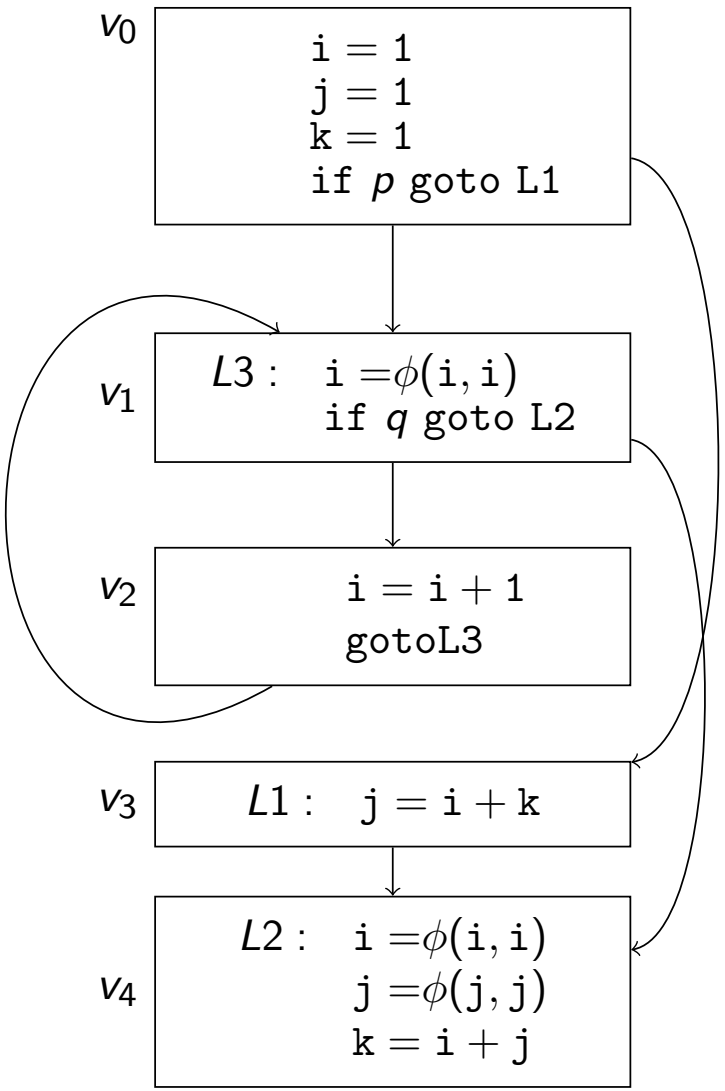
i の定義：$S_0 = \{v_0, v_2\}$
j の定義：$S_0' = \{v_0, v_3\}$
k の定義：$S_0'' = \{v_0, v_4\}$

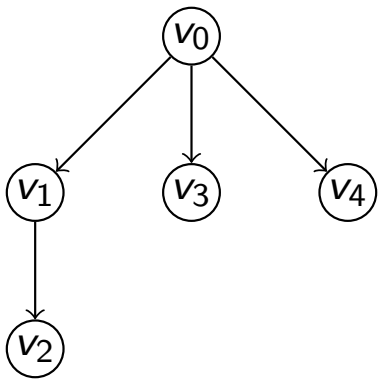
$$DF^+(S_0) = \{v_1, v_4\}$$
$$DF^+(S_0') = \{v_4\}$$
$$DF^+(S_0'') = \emptyset$$

SSA 形式への変換アルゴリズム

1. 各変数 x に対してスタック $\mathrm{stack}(x)$ とカウンタ $\mathrm{count}(x)$ を用意して，各スタックに 0 をプッシュして $\mathrm{count}(x)$ を 0 に初期化
2. 支配木を深さ優先でトラバースして添字を割り当てる．
   - 2.1 φ 関数以外の右辺に各変数のスタックトップの添字を割り当て
   - 2.2 スタックトップの添字を$+1$ した数を push して，左辺の変数にわりあて
   - 2.3 φ 関数の引数に先行頂点の添字のスタックトップを割り当て
   - 2.4 子頂点の添字付け
   - 2.5 スタックを pop する

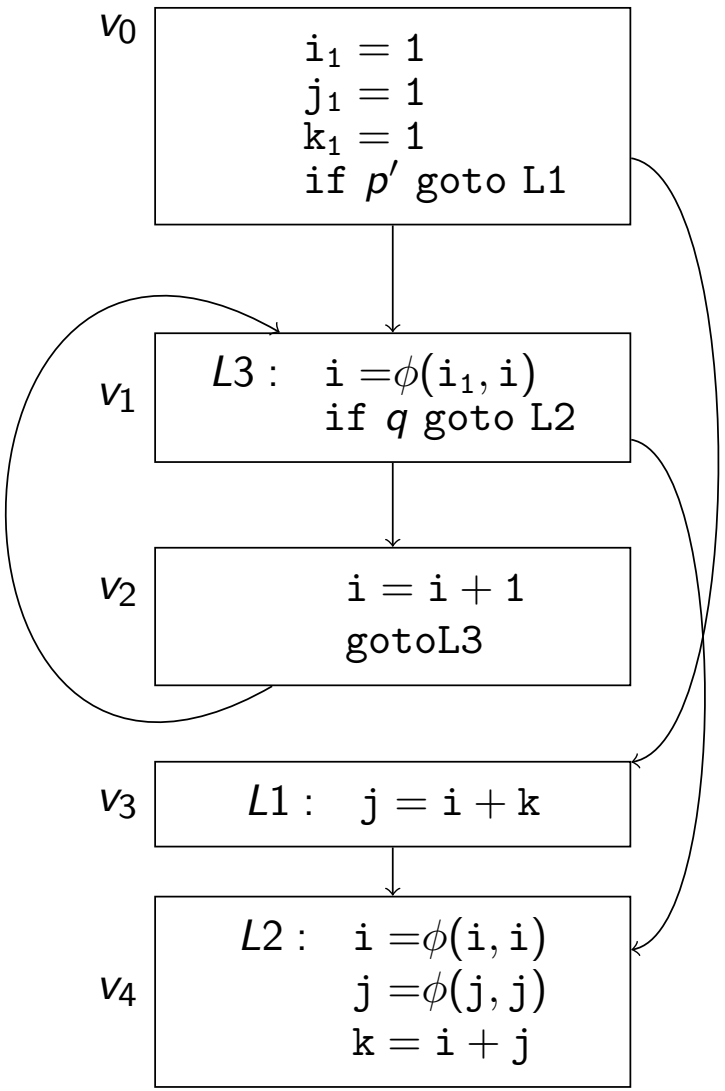
SSA 形式への変換

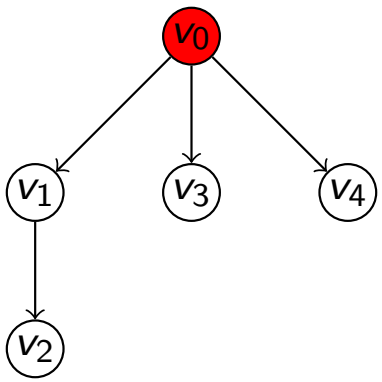




$$stack(i) = [0] \quad c(i) = 0$$
$$stack(j) = [0] \quad c(j) = 0$$
$$stack(k) = [0] \quad c(k) = 0$$

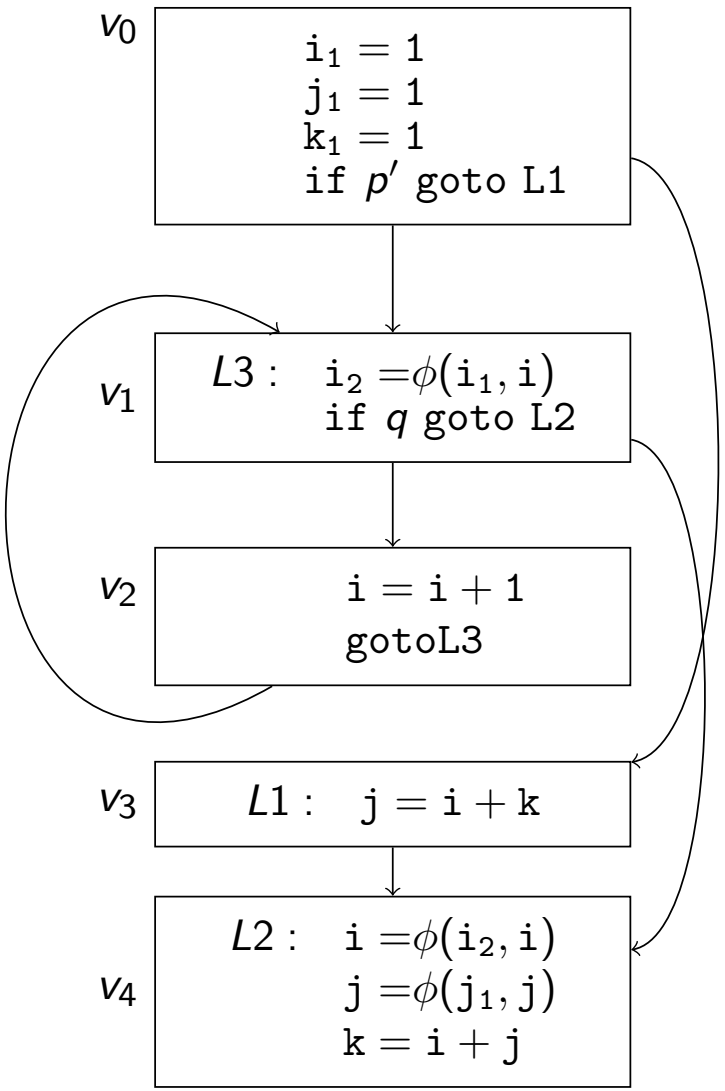
SSA 形式への変換

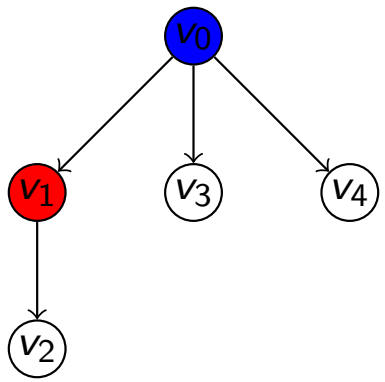




$$stack(i) = [1, 0] \quad c(i) = 1$$
$$stack(j) = [1, 0] \quad c(j) = 1$$
$$stack(k) = [1, 0] \quad c(k) = 1$$

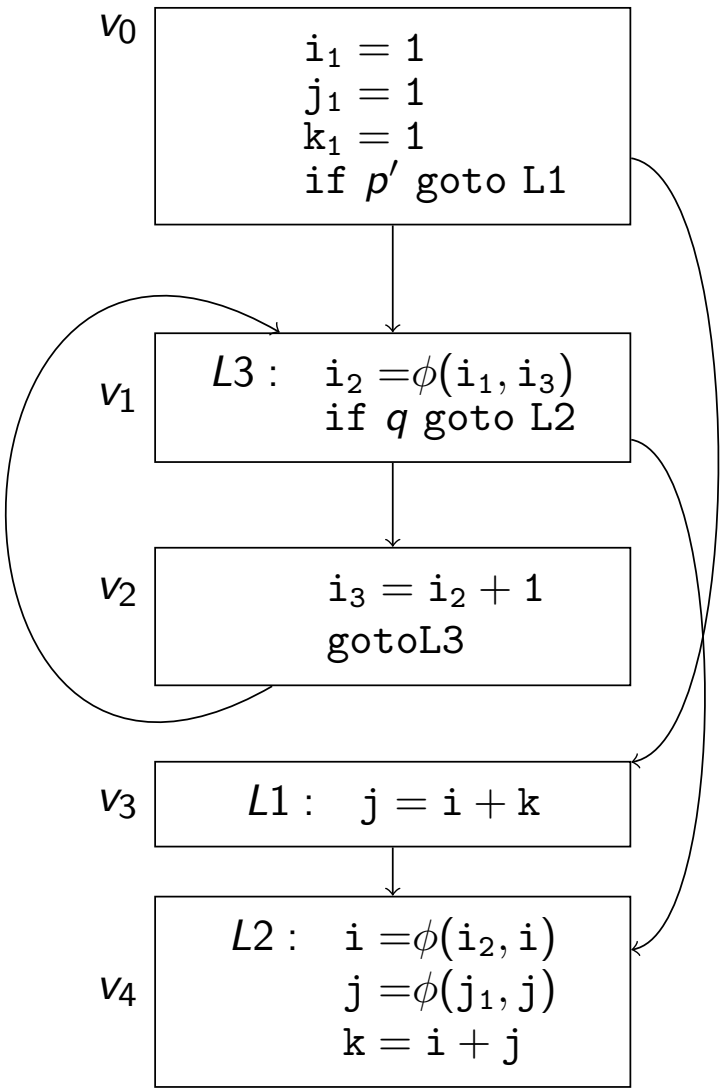
SSA 形式への変換

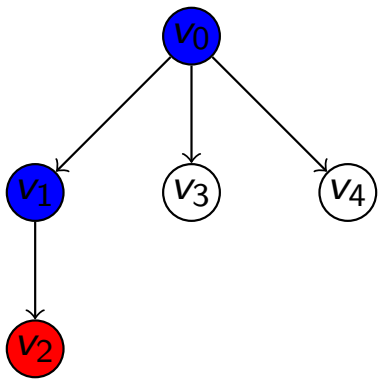




$stack(i) = [2, 1, 0] \quad c(i) = 2$
$stack(j) = [1, 0] \quad c(j) = 1$
$stack(k) = [1, 0] \quad c(k) = 1$

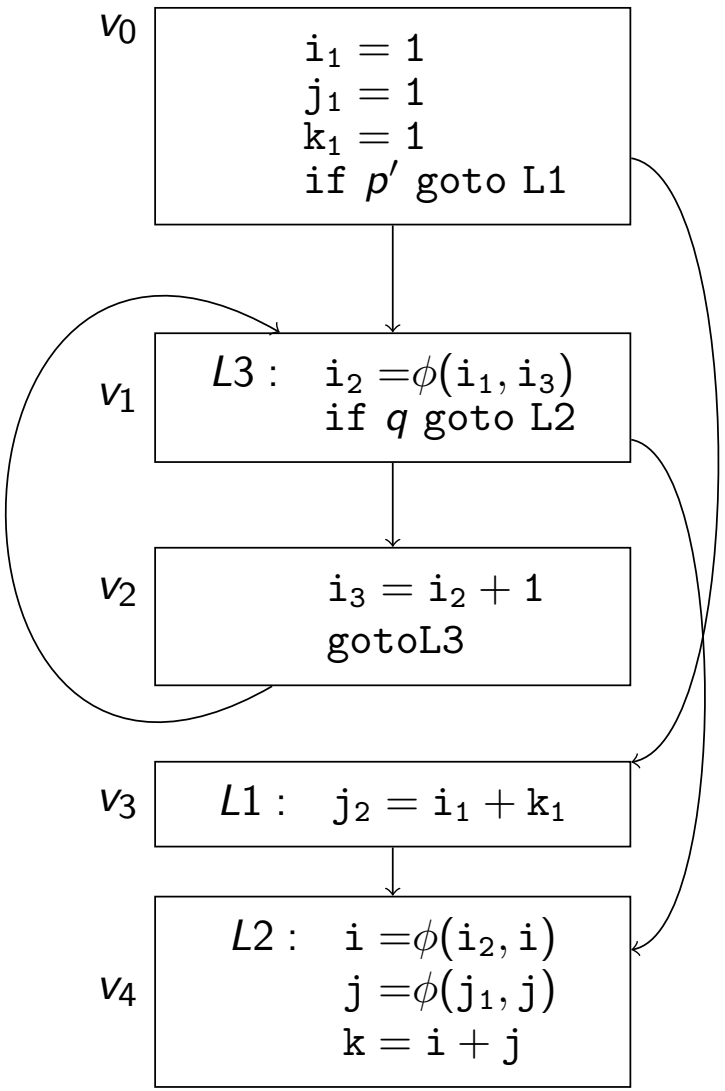
SSA 形式への変換

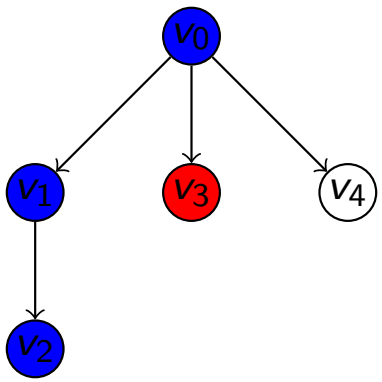




$stack(i) = [3, 2, 1, 0] \quad c(i) = 3$
$stack(j) = [1, 0] \quad c(j) = 1$
$stack(k) = [1, 0] \quad c(k) = 1$

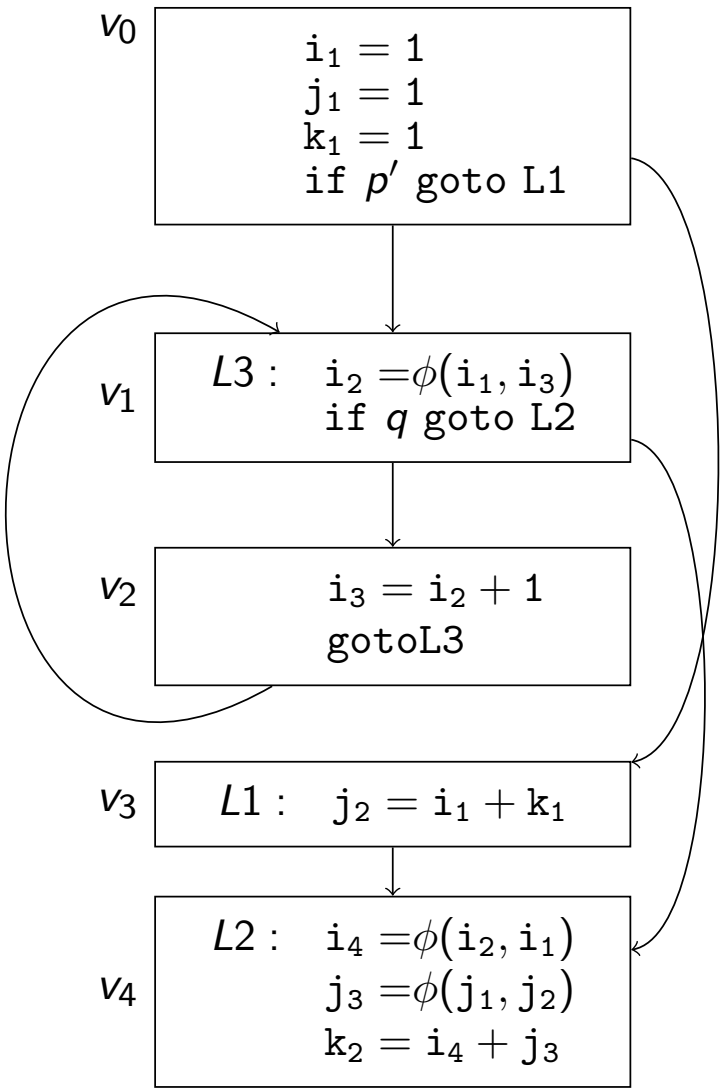
SSA 形式への変換

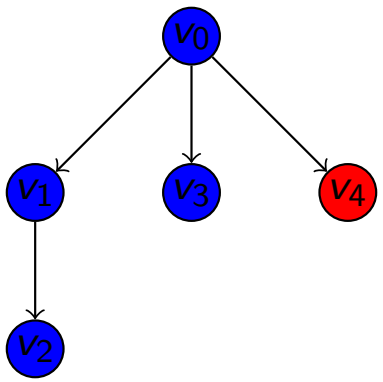




$stack(i) = [1, 0]$ $c(i) = 3$
$stack(j) = [2, 1, 0]$ $c(j) = 2$
$stack(k) = [1, 0]$ $c(k) = 1$

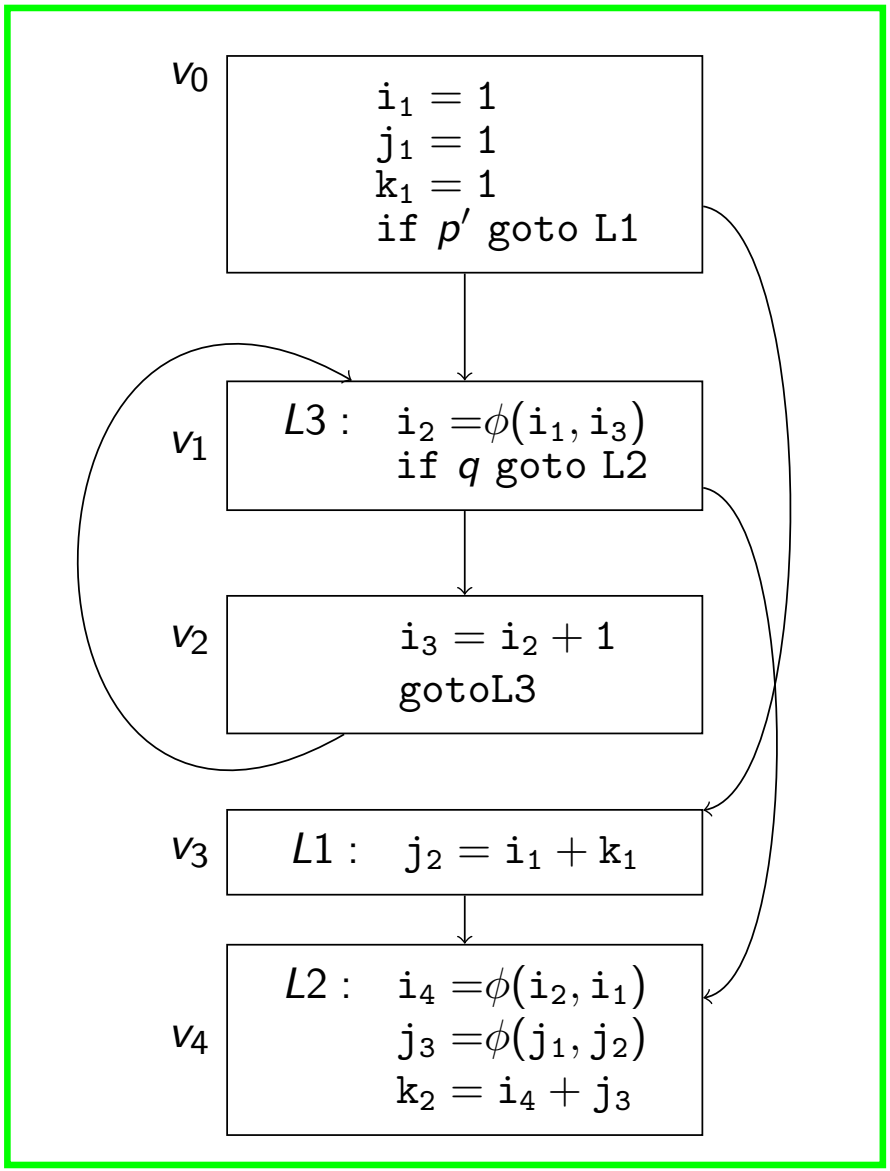
SSA 形式への変換

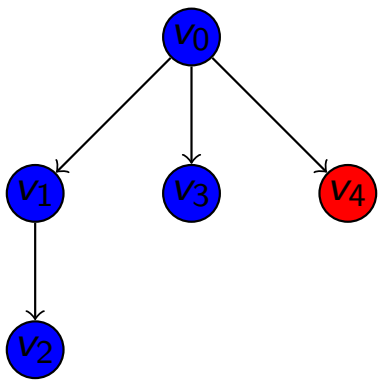




$$stack(i) = [4, 1, 0] \quad c(i) = 4$$
$$stack(j) = [3, 1, 0] \quad c(j) = 3$$
$$stack(k) = [2, 1, 0] \quad c(k) = 2$$

SSA 形式への変換


v_0
$i_1 = 1$
$j_1 = 1$
$k_1 = 1$
if p' goto L1
v_1
$L3:\ i_2 = \phi(i_1, i_3)$
if q goto L2
v_2
$i_3 = i_2 + 1$
gotoL3
v_3
$L1:\ j_2 = i_1 + k_1$
v_4
$L2:\ i_4 = \phi(i_2, i_1)$
$j_3 = \phi(j_1, j_2)$
$k_2 = i_4 + j_3$


v_0
v_1
v_3
v_4
v_2